

Relatório para orientação - alterações após a vetorização da função de sobreposição

Projeto: FundaIA - Otimização do pré-dimensionamento e posicionamento de fundações superficiais

Autor: Lucas Teixeira Correia

Orientação: Profa. Dra. Maria José Pereira Dantas

Data deste relatório: 12/07/2026

Ponto de corte: última versão vista pela orientação: vetorização da verificação de sobreposição entre sapatas, Sprint 3.8.

1. Resumo executivo

Desde a última versão apresentada à orientação, o projeto deixou de ser apenas uma ferramenta com uma função objetivo mais rápida e passou a ter uma base experimental e acadêmica bem mais completa.

As principais mudanças foram:

1. **Arquitetura e rastreabilidade:** o código foi reorganizado em camadas (`core/domain`, `core/engineering`, `core/api`, `core/optimization`, `core/io` e `frontend`), com persistência de experimentos, logs estruturados, cache do modelo substituto e testes automatizados.
2. **Interface:** a aplicação ganhou visualização 2D/3D das sapatas, histórico de convergência do EGO, progresso ao vivo, cancelamento de execução, exportação de artefatos e uma página de experimentos comparando algoritmos.
3. **Engenharia:** foram adicionadas validações de fronteira, corrigida a formulação de tensão no solo, documentada a convenção dos momentos e implementada a verificação de punção também no contorno C' , a 2d da face do pilar.
4. **Metodologia experimental:** foi criado um protocolo com 30 repetições, seeds pareadas, orçamento controlado, comparação com GA, PSO, GWO e busca aleatória, teste pareado de Wilcoxon com correção de Holm, métricas de factibilidade e estudo de penalidade/kernel do GPR.
5. **Otimização com restrições:** foi implementada uma variante de **Constrained Bayesian Optimization (CBO)**, na qual o volume e cada grupo de restrição são modelados separadamente por processos gaussianos.
6. **Artigo:** o manuscrito em `docs/artigo_ic_lucas` foi reposicionado como artigo de **pré-dimensionamento geométrico experimental**, com resultados regeneráveis, referências revisadas, limitações declaradas e texto polido para uma futura submissão.
7. **Fase B:** foi iniciado um piloto de posicionamento/packing, ainda fora do artigo 1, mostrando que quando as sapatas podem se deslocar em planta o problema deixa de ser quase separável.

O ponto mais importante para comunicar é: **o artigo 1 ficou mais honesto e metodologicamente forte, mas o escopo continua sendo pré-dimensionamento geométrico, não projeto executivo completo de fundações.**

2. Linha do tempo resumida das mudanças

Etapa	O que mudou	Por que importa
Sprint 3.8	Vetorização da sobreposição entre sapatas com matriz $N \times N$ em NumPy	Reduziu muito o tempo da função objetivo sem alterar o resultado numérico.
Sprint 4.1	Cache do GPR	Evita treinar o mesmo modelo substituto repetidas vezes quando os dados são idênticos.
Sprint 4.2	Persistência de experimentos	Cada otimização pode salvar configuração, histórico, ambiente, métricas e artefatos.
Sprint 4.3	Reorganização do repositório	Separou código de domínio, engenharia, otimização, IO, API e frontend.
Sprint 4.4	Logs estruturados	Permite auditar execuções por eventos, como <code>ego.iter</code> , <code>cache.hit</code> e <code>optimize.end</code> .
Sprints 4.5-4.11	Interface 3D, tema visual, gráfico de convergência, progresso e cancelamento	Transformou a aplicação em ferramenta mais apresentável e mais fácil de depurar.
Sprint 4.12	Bancada de experimentos	Criou comparação controlada entre EGO, GA, PSO, GWO e depois busca aleatória/CBO.
Sprint 5.1	Protocolo experimental final e guardrails	Fechou casos, seeds, orçamento, métricas de factibilidade e estudo GPR.
Sprint 5.2	Punção no contorno C' e artigo em duas colunas	Corrigiu uma lacuna de verificação estrutural importante para sapatas.
Sprint 5.3	CBO	Separou volume e restrições

Etapa	O que mudou	Por que importa
		em modelos substitutos independentes.
Sprint 5.4	Correção da tensão no solo	Removeu coeficientes legados, incluiu peso próprio real e documentou convenção de momentos.
Sprint 5.5	Novos artigos e decomposição por sapata	Fortaleceu as referências e mostrou que os casos atuais são quase separáveis.
Sprint 5.6	Piloto de packing/layout	Iniciou a fase de posicionamento conjunto, ainda como piloto.
Sprint 5.7	Submission polish	Poliu o artigo, removeu placeholders e deixou o manuscrito-base mais maduro.

3. Explicação didática da função objetivo

O otimizador precisa de uma função que diga se uma solução é boa ou ruim. No FundalA, uma solução é um vetor com as dimensões de cada sapata:

$[h_x, h_y, h_z]$ para cada sapata
onde:

- h_x : largura/comprimento da sapata na direção x;
- h_y : largura/comprimento da sapata na direção y;
- h_z : altura da sapata.

O objetivo principal é reduzir o volume de concreto:

$$\text{Volume} = h_x * h_y * h_z$$

Mas não basta minimizar volume, porque uma sapata muito pequena pode violar critérios de projeto. Por isso usamos uma **função pseudo-objetivo penalizada**:

$$\text{Theta} = \text{Volume} + \text{penalidades}$$

As penalidades entram quando alguma restrição é violada. A convenção usada no código é:

$g \leq 0$ -> restrição atendida
 $g > 0$ -> restrição violada

Atualmente os principais grupos de restrições são:

1. **Sobreposição (g_{sob})** - sapatas não podem ocupar a mesma área em planta.

2. **Punção (g_pun)** - o pilar não pode "furar" a sapata por cisalhamento/punção.
3. **Tensão no solo (g_ten)** - a tensão transmitida ao solo não pode superar a tensão admissível, e a interface solo-sapata não deve ficar tracionada.
4. **Geometria (g_geo)** - a sapata precisa ter balanço mínimo ao redor do pilar.

Arquivos principais:

- `core/api/objective.py`
 - `core/engineering/tensao.py`
 - `core/engineering/puncao.py`
 - `core/engineering/packing.py`
-

4. Vetorização da sobreposição - o que a orientação já tinha visto

Antes, a verificação de sobreposição fazia comparações par a par usando laços Python e `df.iterrows()`. Isso funcionava, mas era lento quando havia muitas sapatas.

Depois da Sprint 3.8, a sobreposição passou a ser calculada por uma matriz $N \times N$. Cada célula da matriz representa a área de interseção entre duas sapatas.

Em termos simples:

1. cada sapata é tratada como um retângulo em planta;
2. o código calcula quanto os retângulos se sobrepõem no eixo x;
3. calcula quanto se sobrepõem no eixo y;
4. multiplica os dois valores para obter área de sobreposição;
5. zera a diagonal da matriz, porque uma sapata não deve ser comparada com ela mesma.

O resultado é o mesmo da versão antiga, mas muito mais rápido. A validação da Sprint 3.8 registrou igualdade bit a bit contra a versão escalar e speedup de aproximadamente 100x a 160x em casos maiores.

Arquivo principal:

- `core/engineering/packing.py::sobreposicao_matrix`
-

5. Correção da tensão no solo

5.1. Qual era o problema

Na formulação anterior havia coeficientes legados (1,05 e 1,30) e o peso próprio da sapata não aparecia claramente como função do volume. Isso era fraco do ponto de vista físico e difícil de defender no artigo.

Também havia risco de confusão na convenção dos momentos M_x e M_y .

5.2. Como ficou agora

A tensão na base da sapata é calculada como flexão composta:

$\sigma = \text{carga vertical} / \text{area} \pm \text{termos de momento}$

No código atual:

```
area = h_x * h_y
peso_proprio = gamma_c * h_x * h_y * h_z
sigma_axial = (F_z + peso_proprio) / area
sigma_mx = 6 * |M_x| / (area * h_x)
sigma_my = 6 * |M_y| / (area * h_y)

sigma_max = sigma_axial + sigma_mx + sigma_my
sigma_min = sigma_axial - sigma_mx - sigma_my
```

Foi adotado:

$\gamma_c = 25 \text{ kN/m}^3$

Ou seja, se a sapata fica maior ou mais alta, o peso próprio aumenta automaticamente. Isso é coerente com a física, porque uma sapata maior pesa mais e transmite mais carga ao solo.

5.3. Convenção de momentos

No FundalA ficou documentado que:

$M_x = F_z * e_x$
 $M_y = F_z * e_y$

Isso quer dizer:

- M_x é o momento associado à excentricidade na direção x ;
- M_y é o momento associado à excentricidade na direção y .

Importante: se uma planilha externa vier de um software estrutural que usa "momento em torno do eixo X" e "momento em torno do eixo Y", pode ser necessário converter os eixos antes de importar. Essa observação foi colocada no código e no artigo.

Arquivo principal:

- `core/engineering/tensao.py`

6. Implementação da punção no contorno C'

6.1. O que é punção, em linguagem simples

Punção é uma forma de ruptura em que o pilar tende a "perfurar" a sapata, como se empurrasse um bloco de concreto para baixo. Em sapatas e lajes, a NBR 6118 trata a punção por contornos críticos ao redor do pilar.

Antes o projeto verificava apenas o contorno **C**, na face do pilar. Esse contorno está ligado ao esmagamento da biela comprimida.

Foi implementado também o contorno **C'**, localizado a $2d$ da face do pilar, onde:

$$d = h_z - \text{cobrimento}$$

Em palavras simples, d é a altura útil da sapata: altura total menos cobrimento.

6.2. O que foi implementado

Agora o grupo de punção considera o pior resultado entre:

$$g_{\text{puncao}} = \max(g_C, g_{C'})$$

Se qualquer um dos dois contornos violar, a sapata é tratada como violada.

No contorno **C'**:

- o perímetro fica a $2d$ da face do pilar;
- os momentos entram em módulo, para não reduzir artificialmente a solicitação;
- usa-se coeficiente K da Tabela 19.2 da NBR 6118;
- usa-se taxa mínima de armadura $\rho_{\min}$ da Tabela 17.3 da NBR 6118;
- a reação do solo dentro do perímetro não é abatida, decisão conservadora e coerente com a leitura adotada para a NBR.

6.3. Resultado prático nos casos estudados

A implementação foi testada e, nos três casos congelados do artigo, o contorno **C'** ficou folgado.

Portanto:

- a verificação ficou mais completa;
- mas ela não mudou os melhores resultados numéricos do protocolo nesses casos;
- a conclusão do artigo passou a ser: punção **C** e **C'** foram verificadas, mas não governaram os casos estudados.

Fontes principais:

- ABNT NBR 6118, item 19.5, tabelas 17.3 e 19.2;
- Santos, Lima Neto e Ferreira (2018), artigo sobre resistência a punção em sapatas de concreto armado.

Arquivos principais:

- `core/engineering/puncao.py`
- `core/api/objective.py`
- `tests/test_engenharia.py`

7. Validacoes de fronteira adicionadas

Foram adicionadas protecoes para impedir que o código aceite situações fisicamente sem sentido.

7.1. $F_z > 0$

A carga vertical deve ser positiva. Carga nula ou tração/uplift não está dentro do modelo atual de contato solo-sapata comprimido.

7.2. $h_z > \text{cobrimento}$

A altura útil é:

$$d = h_z - \text{cobrimento}$$

Se $h_z \leq \text{cobrimento}$, então $d \leq 0$, o que faria fórmulas de punção ficarem erradas e poderia até fazer uma sapata impossível parecer viável.

7.3. f_{ck} em kPa

O projeto espera f_{ck} em kPa, por exemplo:

$$25 \text{ MPa} = 25.000 \text{ kPa}$$

Foi adicionada validação para evitar que alguém informe 25 pensando em MPa. Isso geraria resistência de concreto muito errada.

Arquivos principais:

- `core/domain/combinacao.py`
 - `core/domain/projeto.py`
 - `core/api/objective.py`
 - `core/engineering/puncao.py`
-

8. Cache do GPR, persistência e logs

8.1. Cache do GPR

O EGO usa um modelo substituto chamado GPR. Esse modelo precisa ser treinado várias vezes. Se os mesmos dados e a mesma configuração aparecem de novo, o cache reaproveita o modelo treinado em vez de recalcular.

Isso não muda o resultado matemático. O cache só entra quando a "impressão digital" dos dados e da configuração é exatamente a mesma.

Arquivo principal:

- `core/optimization/cache.py`

8.2. Persistência de experimentos

Agora cada execução pode salvar uma pasta com:

- configuração usada;
- versões de bibliotecas;
- entrada do projeto;

- histórico por repetição;
- resumo em CSV;
- métricas agregadas;
- artefatos como figuras e tabelas.

Isso é importante porque o artigo não depende mais de números copiados manualmente. As tabelas e figuras podem ser regeneradas a partir dos artefatos.

Arquivo principal:

- `core/io/experiments.py`

8.3. Logs estruturados

Os logs agora podem sair como linhas JSON, com eventos nomeados. Isso ajuda a entender o que aconteceu durante uma execução longa.

Exemplos de eventos:

- `optimize.start`
- `lhs.done`
- `ego.iter`
- `cbo.iter`
- `experiment.end`

Arquivo principal:

- `core/observability/logging.py`
-

9. Interface e ferramentas visuais

A interface Streamlit foi bastante ampliada.

9.1. Visualizacao 3D

Agora é possível ver as sapatas como blocos 3D:

- sapata enterrada;
- pilar acima;
- plano do solo;
- tooltip com dimensões e volume;
- rotação e zoom interativos.

Isso ajuda a explicar visualmente o resultado ao orientador e a verificar se alguma solução parece estranha.

Arquivo principal:

- `frontend/components/footings_3d.py`

9.2. Histórico do EGO

A interface mostra a curva de convergência: como o melhor valor de Theta vai melhorando ao longo das iterações.

Arquivo principal:

- `frontend/components/ego_chart.py`

9.3. Progresso ao vivo e cancelamento

Durante a otimização, a UI mostra:

- repetição atual;
- iteração atual;
- melhor valor encontrado;
- fase atual: LHS, EGO, gravacao etc.

Também foi adicionado cancelamento cooperativo.

Arquivos principais:

- `core/api/optimize.py`
- `core/optimization/ego.py`
- `frontend/pages/sapatas.py`

9.4. Página de experimentos

Foi criada uma página própria para comparação científica de algoritmos. Ela roda EGO, CBO, GA, PSO, GWO e busca aleatória com orçamento controlado e gera:

- curva de convergência;
- tabela resumo;
- matriz de p-valores Wilcoxon-Holm;
- bundle de resultados.

Arquivos principais:

- `core/api/benchmark.py`
 - `frontend/pages/experimentos.py`
 - `frontend/components/convergence_chart.py`
-

10. Protocolo experimental do artigo

O protocolo do artigo foi fechado para evitar comparações injustas.

10.1. Casos de estudo

Foram usados três casos congelados:

- Caso 1: uma sapata, dimensão de busca 3;
- Caso 2: duas sapatas, dimensão de busca 6;
- Caso 3: três sapatas, dimensão de busca 9.

Os limites adotados no artigo foram:

$h_{min} = 0,60 \text{ m}$
 $h_{max} = 3,00 \text{ m}$
 $f_{ck} = 25 \text{ MPa}$
 $cobrimento = 0,04 \text{ m}$

10.2. Cenário S1 - orçamento igual

Todos os algoritmos usam 150 avaliações reais por repetição, com 30 repetições pareadas.

Esse cenário responde:

qual algoritmo consegue bons resultados com poucas avaliações reais?

10.3. Cenário S2 - orçamento estendido

As buscas diretas recebem 3.000 avaliações.

Esse cenário responde:

se a função objetivo for barata, o que acontece quando damos muito orçamento aos algoritmos diretos?

10.4. Estatística

As repetições usam seeds pareadas. Por isso a comparação estatística passou a usar:

teste pareado de Wilcoxon + correção de Holm

Isso é mais correto que comparar amostras como se fossem independentes.

11. Penalidade, GPR e kernels

11.1. O que foi testado

O estudo de penalidade avaliou como o fator de penalidade afeta o GPR.

Foram comparados, por exemplo:

$\alpha = 10$
 $\alpha = 10^6$

11.2. Resultado importante

Um resultado forte foi:

- penalidade muito alta não necessariamente derruba o R^2 global;
- mas aumenta enormemente o erro na região factível.

Em termos simples: o modelo pode parecer bom olhando a metrica global, mas ficar ruim justamente perto das soluções que interessam para o projeto.

Isso justificou manter penalidade moderada e também motivou o CBO.

11.3. Kernels

Foram testadas 21 configurações de kernel do GPR. O resultado foi que a escolha fina do kernel pareceu menos importante que a escala da penalidade para esta formulação.

12. CBO - otimização bayesiana com restrições

12.1. Por que implementar CBO

No EGO penalizado, o GPR aprende uma função que mistura:

volume + penalidades

Isso pode ser ruim porque a penalidade cria uma descontinuidade ou variação artificial muito forte. O modelo substituto passa a gastar capacidade aprendendo essa penalidade, em vez de aprender apenas o comportamento físico do volume.

No CBO, a ideia é separar:

1 GP para o volume

1 GP para cada grupo de restrição

Depois, a função de aquisição escolhe pontos que prometem melhorar o volume e, ao mesmo tempo, tem alta probabilidade de serem factíveis.

12.2. Formula conceitual

A aquisição usada segue a ideia de Gardner et al. (2014):

$$ECI(x) = EI(\text{volume}) * P(\text{restrição 1 viável}) * \dots * P(\text{restrição k viável})$$

Se nenhum ponto factível ainda foi encontrado, o método tenta primeiro maximizar a probabilidade de factibilidade.

12.3. Resultado do artigo

Depois da correção da tensão e da regeneração dos resultados:

- CBO melhorou a media de Theta em relação ao EGO nos três casos;
- também melhorou o melhor volume estritamente factível;
- mas perdeu factibilidade estrita nos dois casos menores.

Por isso a leitura correta não é "CBO é sempre melhor". A leitura correta é:

CBO é uma alternativa metodologicamente promissora para tratar restrições, mas precisa de regras finais de seleção factível e deve ser validado em problemas realmente acoplados.

Fontes principais:

- Gardner et al. (2014), Bayesian Optimization with Inequality Constraints;
- Eriksson e Poloczek (2021), Scalable Constrained Bayesian Optimization;
- Mathern et al. (2021), CBO em projeto estrutural.

Arquivos principais:

- `core/optimization/cbo.py`
- `core/api/objective.py::avaliar_projeto_componentes`
- `core/api/benchmark.py`
- `tests/test_cbo.py`

13. Auditoria de decomposição por sapata

Uma crítica importante ao artigo era que os casos atuais poderiam ser quase separáveis.

Quase separável significa:

otimizar cada sapata isoladamente e juntar os resultados da quase o mesmo que otimizar tudo junto.

Isso acontece porque, nos três casos congelados, a restrição de sobreposição fica inativa por construção. As sapatas estão longe o suficiente para não "brigar" por espaço.

Para medir isso, foi implementado um baseline de Differential Evolution por sapata:

1. otimiza uma sapata por vez;
2. junta os resultados;
3. reavalia tudo no avaliador global.

Resultado:

Caso	Volume decomposto	Melhor protocolo global	Ganho da decomposição
Caso 1	3,108824 m3	3,108826 m3	<0,01%
Caso 2	4,750747 m3	4,787486 m3	0,77%
Caso 3	2,122252 m3	2,167259 m3	2,08%

Conclusão: os casos atuais são bons para testar metodologia, penalidade, CBO e protocolo, mas ainda não demonstram plenamente o ganho em problemas acoplados de posicionamento.

Arquivo principal:

- `scripts/run_decomposition_baseline.py`

14. Fase B - piloto de posicionamento e packing

14.1. Por que iniciar a Fase B

Depois da auditoria de decomposição, ficou claro que o artigo 1 ainda não testa fortemente o acoplamento entre sapatas. O acoplamento aparece quando as posições também viram variáveis de projeto e as sapatas podem se aproximar, encostar em limites do lote ou se sobrepor.

14.2. O que foi testado no piloto

Foi criado um caso sintético com duas sapatas próximas, derivado da planilha `problema_fund_dois.xlsx`.

No piloto, cada sapata pode ter:

h_x , h_y , h_z , dx , dy

onde:

- dx : deslocamento do centro da sapata em relação ao pilar no eixo x;
- dy : deslocamento no eixo y.

Quando a sapata é deslocada, os momentos efetivos são ajustados:

$$M_{x_eff} = M_{x_input} - F_z * dx$$

$$M_{y_eff} = M_{y_input} - F_z * dy$$

O piloto também adiciona restrições:

- o pilar precisa continuar dentro da sapata;
- a sapata precisa ficar dentro dos limites do lote;
- as sapatas não podem se sobrepor.

14.3. Resultado do piloto

Modo	Volume	Sobreposição	Factive!	Interpretação
Otimos individuais centralizados	4,750747 m3	$g_{sob} = 0,2307$	Não	Cada sapata fica boa isoladamente, mas juntas se sobrepõem.
Centros fixos, redimensionando	4,929703 m3	$g_{sob} = 0$	Sim	Fica factível, mas precisa de mais volume.
Com deslocamentos dx , dy	4,525122 m3	$g_{sob} = 0$	Sim	Permitir posicionamento reduziu o volume factível.

Esse resultado ainda não é evidência estatística para artigo. Ele serve como prova de conceito de que a Fase B é relevante.

Arquivo principal:

- `scripts/run_packing_phase_b_pilot.py`
-

15. Artigo em `docs/artigo_ic_lucas`

O artigo foi bastante revisado.

Mudanças principais:

- passou para formato de duas colunas;
- incluiu figura dos arranjos em planta;
- incluiu punção C e C';
- incluiu CBO;
- incluiu estudo de penalidade e kernels;
- incluiu protocolo S1/S2;
- incluiu Wilcoxon pareado com Holm;
- incluiu auditoria de decomposição;
- removeu notas internas;
- deixou claro que o escopo é pré-dimensionamento geométrico, não projeto executivo;
- fechou agradecimentos, conflitos de interesse e disponibilidade de dados/código de forma genérica;
- foi polido para reduzir termos hermeticos.

O PDF final compilado tem 22 páginas:

- arquivo: `docs/artigo_ic_lucas/main.pdf`;
 - sem erro de compilacao;
 - sem referências/citacoes indefinidas;
 - sem avisos críticos de `Overfull` após a rodada de polish.
-

16. O que ainda não está resolvido

Esses pontos devem ser apresentados como pendencias honestas, não como falhas escondidas.

16.1. Correlacao N_{spt} - tensão admissível

O código ainda usa uma correlação empírica simples:

pedregulho: N_{spt} / 30
areia: N_{spt} / 40
outros: N_{spt} / 50

Ela está tratada como hipótese preliminar de pré-dimensionamento. Antes de submissão forte, o ideal é validar ou substituir por uma referência geotécnica mais robusta.

16.2. Combinações de ações

A separação formal entre combinações de serviço e estado limite último ainda precisa ser amadurecida.

16.3. Flexão, cisalhamento unidirecional e armadura

O artigo atual não dimensiona armadura, não verifica flexão/cisalhamento unidirecional, não trata ancoragem/detalhamento e não calcula custo total. Isso foi deixado como trabalho futuro porque incluir tudo agora mudaria o escopo para projeto executivo completo.

16.4. Fase B ainda é piloto

O packing/layout já foi iniciado, mas ainda precisa virar protocolo experimental pareado, com vários casos acoplados congelados e comparação estatística.

16.5. Forma exata da referência da NBR 6118

O catálogo ABNT indica a existência da ABNT NBR 6118:2026. Mesmo assim, antes de submissão formal, recomenda-se conferir no acesso institucional/ABNT Coleção se a forma bibliográfica correta deve ser citada como norma 2026, como NBR 6118:2023 com Emenda 1:2026, ou como versão corrigida/emendada.

17. Como eu explicaria para a orientadora em 2 minutos

Uma forma curta de apresentar:

Depois da vetorização da sobreposição, reorganizamos o projeto para ficar mais reproduzível e defensável cientificamente. A função objetivo foi consolidada em uma versão vetorizada, com testes de regressão, e acrescentamos validações para impedir entradas fisicamente inválidas. Na parte de engenharia, corrigimos a tensão no solo para usar peso próprio real da sapata e documentamos a convenção dos momentos. Também completamos a verificação de punção com o contorno C' a 2d, baseado na NBR 6118 e em Santos et al. (2018).

Na parte metodológica, criamos um protocolo experimental com 30 repetições pareadas, orçamento controlado, comparação com GA, PSO, GWO, busca aleatória e CBO, usando Wilcoxon pareado com correção de Holm. O CBO foi implementado para modelar volume e restrições separadamente, porque a penalização pode deformar o modelo substituto. Também fizemos uma auditoria por decomposição que mostrou que os casos atuais são quase separáveis, então reposicionamos o artigo como pré-dimensionamento geométrico experimental, com a Fase B de packing como próximo passo.

O artigo foi polido, compila e já tem as limitações explicitadas. O que falta antes de uma submissão mais forte é validar melhor a correlação N_{spt}-tensão admissível, escolher o

template do evento/revista e, se desejarmos outro artigo ou uma seção futura, transformar o piloto de packing em protocolo completo.

18. Arquivos principais para revisão

Código de engenharia

- `core/engineering/tensao.py`
- `core/engineering/puncao.py`
- `core/engineering/packing.py`
- `core/api/objective.py`

Otimização e experimentos

- `core/optimization/ego.py`
- `core/optimization/cbo.py`
- `core/api/benchmark.py`
- `core/io/experiments.py`
- `scripts/run_final_benchmark.py`
- `scripts/run_cbo_benchmark.py`
- `scripts/run_decomposition_baseline.py`
- `scripts/run_packing_phase_b_pilot.py`
- `scripts/make_paper_artifacts.py`

Interface

- `frontend/pages/sapatas.py`
- `frontend/pages/experimentos.py`
- `frontend/components/footings_3d.py`
- `frontend/components/ego_chart.py`
- `frontend/components/convergence_chart.py`

Artigo e documentação

- `docs/artigo_ic_lucas/main.tex`
- `docs/artigo_ic_lucas/secoes/04_metodologia.tex`
- `docs/artigo_ic_lucas/secoes/06_resultados_parciais.tex`
- `docs/artigo_ic_lucas/secoes/07_discussao.tex`
- `docs/artigo_ic_lucas/secoes/08_conclusoes_parciais.tex`
- `docs/artigo_ic_lucas/main.pdf`
- `obsidian_vault/12_Auditoria/`

19. Fontes usadas como base técnica

Normas e referências de engenharia

- ABNT NBR 6118 - Projeto de estruturas de concreto - Procedimento. Usada para a verificação de punção nos contornos C e C', incluindo Tabelas 17.3 e 19.2. Observação: conferir forma bibliográfica exata antes de submissão.
- ABNT NBR 6122 - Projeto e execução de fundações. Usada como referência geral de fundações superficiais.
- Santos, D. F. A.; Lima Neto, A. F.; Ferreira, M. P. (2018). *Punching shear resistance of reinforced concrete footings: evaluation of design codes*. Revista IBRACON de Estruturas e Materiais. DOI: 10.1590/S1983-41952018000200011.
- Wang, Y.; Kulhawy, F. H. (2008). *Economic Design Optimization of Foundations*. DOI: 10.1061/(ASCE)1090-0241(2008)134:8(1097).
- Nigdeli, S. M.; Bekdas, G.; Yang, X.-S. (2018). *Metaheuristic Optimization of Reinforced Concrete Footings*. DOI: 10.1007/s12205-018-2010-6.
- Waheed et al. (2022, 2025). Trabalhos sobre ferramentas e otimização econômica de sapatas isoladas de concreto armado.

Otimização, GPR e CBO

- Jones, Schonlau e Welch (1998). *Efficient Global Optimization of Expensive Black-Box Functions*. DOI: 10.1023/A:1008306431147.
- Williams e Rasmussen (1995). *Gaussian Processes for Regression*.
- Shahriari et al. (2016). *Taking the Human Out of the Loop: A Review of Bayesian Optimization*. DOI: 10.1109/JPR0C.2015.2494218.
- Schulz, Speekenbrink e Krause (2018). *A Tutorial on Gaussian Process Regression*. DOI: 10.1016/j.jmp.2018.03.001.
- Gardner et al. (2014). *Bayesian Optimization with Inequality Constraints*.
- Eriksson e Poloczek (2021). *Scalable Constrained Bayesian Optimization*.
- Mathern et al. (2021). *Multi-objective Constrained Bayesian Optimization for Structural Design*. DOI: 10.1007/s00158-020-02720-2.

Geotecnia baseada em dados e trabalhos futuros

- Ahmad et al. (2021). GPR para capacidade de carga de fundações superficiais. DOI: 10.3390/app112110317.
- Khajehzadeh, Keawsawasvong e Nehdi (2022). Otimização de fundações superficiais com soft computing. DOI: 10.3390/su14031847.

- Fattahi, Ghaedi e Armaghani (2025). Predição de recalques com técnicas inteligentes. DOI: 10.32604/cmes.2025.062390.
 - Yu, Picard e Ahmed (2025). Modelos pré-treinados para BO com restrições em problemas de engenharia. DOI: 10.1007/s00158-025-03987-z.
-

20. Revisão de coerencia do relatório

Conferências feitas ao preparar este documento:

- O ponto de corte foi confirmado no histórico: Sprint 3.8, commit `fa95cc195`, vetorização da sobreposição.
- As mudanças posteriores foram reconstruídas a partir de commits, notas do vault, README, artigo e código atual.
- As explicações de tensão, punção C/C', CBO, decomposição e packing foram conferidas contra os arquivos de implementação.
- Os resultados numéricos citados foram retirados das notas de auditoria das Sprints 5.4, 5.5 e 5.6, que documentam reruns e artefatos.
- O relatório distingue implementação consolidada, evidência experimental do artigo e piloto futuro, para não misturar escopos.